

21

Lecture 21

**Memory Management III: TLB and I/O
Introduction**

By Brijesh Shukla
C07: Operating Systems

2

Join the lecture online on your dashboard

Today's Agenda

- Review of Paging and Translation
- Translation Lookaside Buffer (TLB)
- TLB Performance and Management
- Introduction to I/O Management
- Polling vs Interrupts
- Direct Memory Access (DMA)



Review – Why Paging?

❌ Problems with Contiguous Allocation

- External fragmentation wastes memory
- Large processes need contiguous space
- Compaction is costly and disruptive

✅ Paging: The Solution

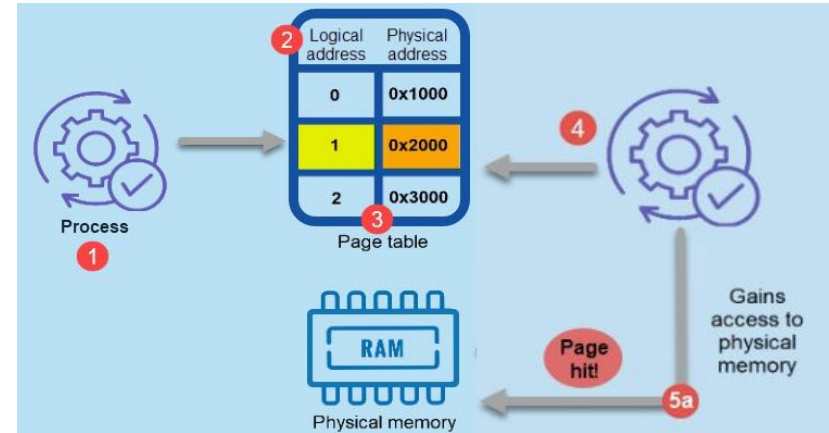
- Physical memory → **fixed-size frames** (e.g., 4KB)
- Logical memory → **fixed-size pages**
- Any page → any free frame

🚀 Why Paging Works

- **No external fragmentation**
- Simple, flexible allocation
- Supports non-contiguous memory
- Efficient memory utilization

🔑 Key Advantage

Logical and physical memory are separated
enabling modern virtual memory systems.



Review – Address Translation in Paging

Virtual → Physical Address

Virtual Address Format:

[Page Number (p) | Offset (d)]

Translation Steps

1. CPU generates virtual address (p, d)
2. p indexes the **Page Table**
3. Page Table gives **Frame number (f)**
4. Physical Address = (f, d)

Example

- Page size = **4KB (4096 bytes)**
- Offset bits = **12**

Virtual Address = **10876**

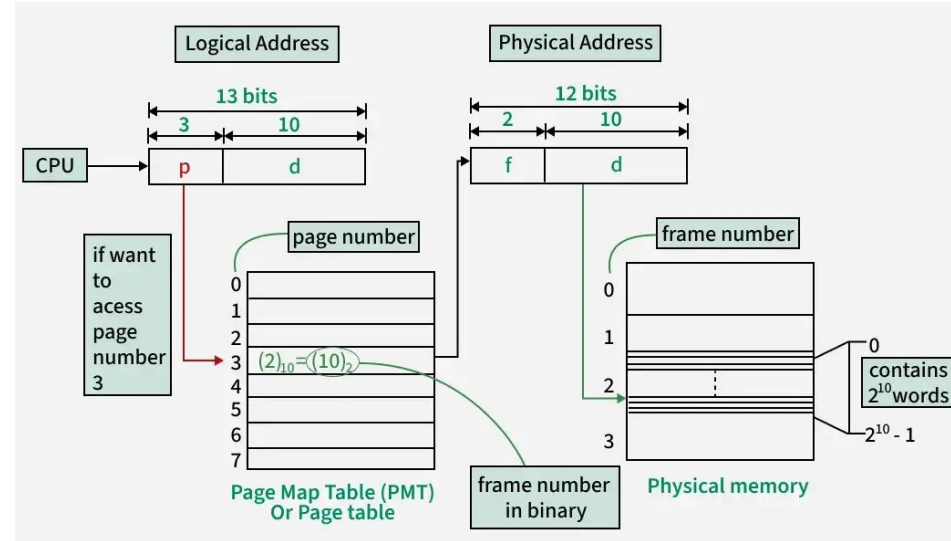
- Page = $10876 / 4096 = 2$
- Offset = $10876 \% 4096 = 2684$

If **Page 2** → **Frame 5**

- Physical Address = $(5 \times 4096) + 2684 = 23164$

Key Takeaway

Paging translates addresses by **table lookup + offset**, enabling flexible, non-contiguous memory allocation.



Review – Page Table Structure

Page Table Entry (PTE)

Each virtual page has one **PTE** describing its state.

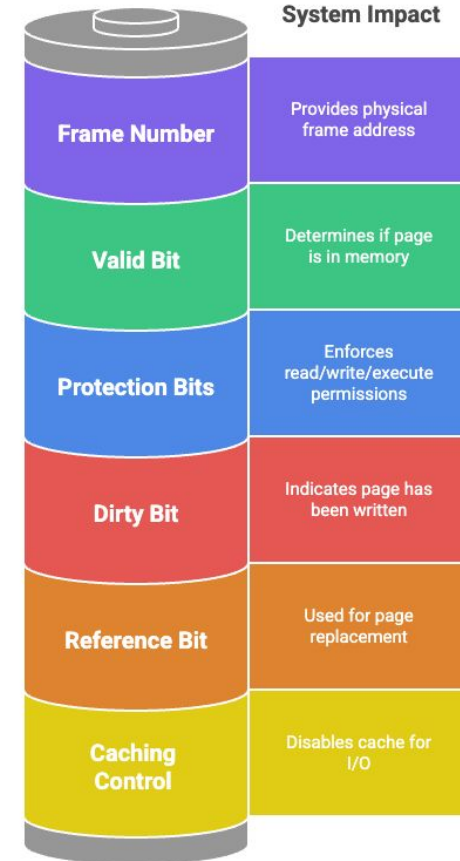
Typical Fields

Why These Fields Matter

- **Correctness:** Valid + protection bits
- **Performance:** Reference bit (replacement), caching control
- **Consistency:** Dirty bit (write-back decisions)

Where the Page Table Lives

- Stored in **main memory**
- Base address held in **PTBR (Page Table Base Register)**.



The Performance Problem

🕒 Why Paging Seems Slow

For **every** memory access, paging needs:

- ① Access **Page Table**
- ② Access **Actual Data**

➔ **Two memory accesses per instruction**

🕒 Example

- Memory access time = **100 ns**

Effective Access Time: 100 ns (page table) +
100 ns (data) = 200 ns

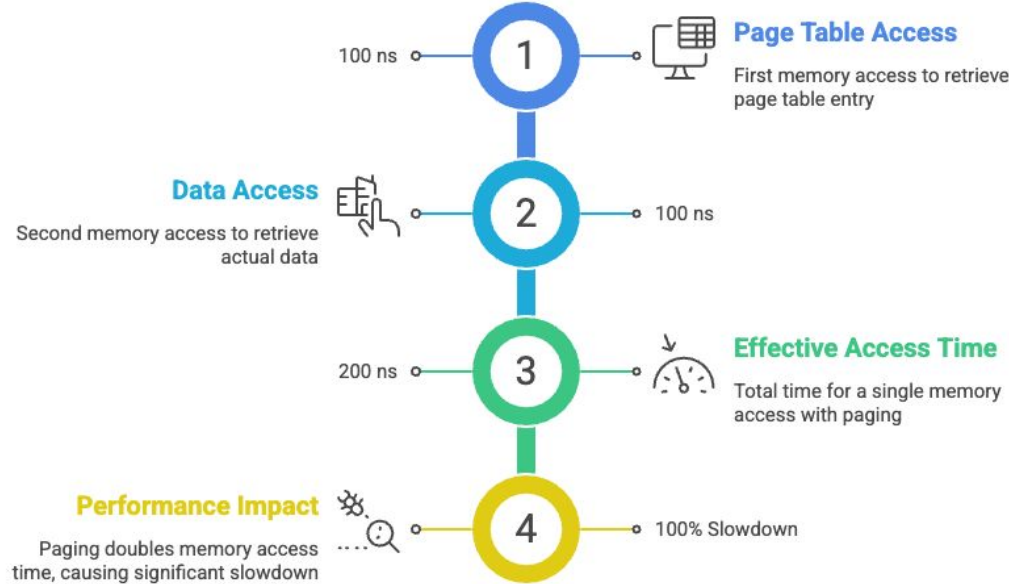
➔ **100% slowdown** ❌

⚠️ Impact

- High latency
- Poor CPU utilization
- Unacceptable for modern systems

🔑 Conclusion

Paging needs a **faster address translation mechanism** to be practical.



Translation Lookaside Buffer (TLB)

🧠 What Is a TLB?

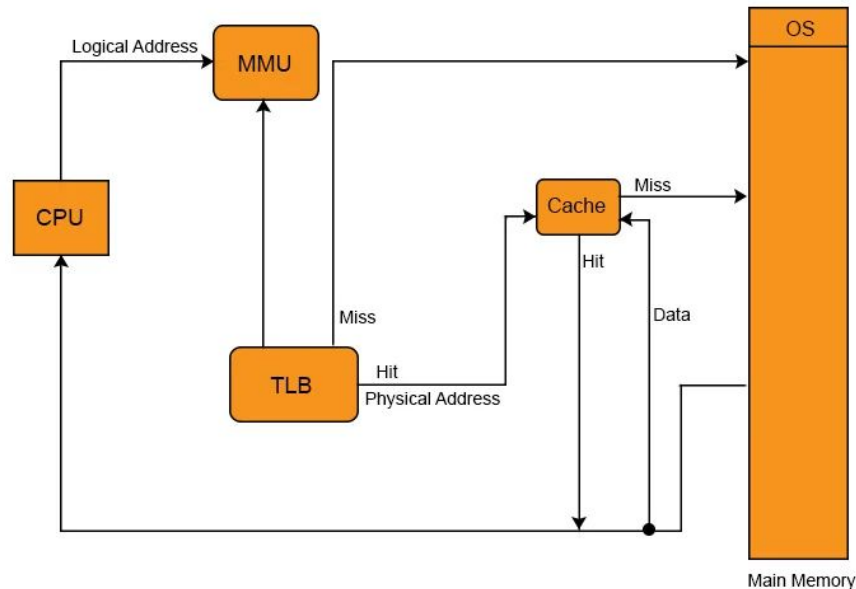
- A **small, very fast cache** for page table entries
- Also called **Address Translation Cache**
- Located inside the **MMU**

⚙️ Key Features

- Stores **recent page** → **frame mappings**
- **Fully / Set associative** (parallel search)
- **1 CPU cycle access** ($< 1 \text{ ns}$)
- Small size (**~64–256 entries**)

🚀 Why TLB Matters

- Avoids frequent page table memory access
- Reduces address translation from **2 accesses** → **1**
- Dramatically improves paging performance



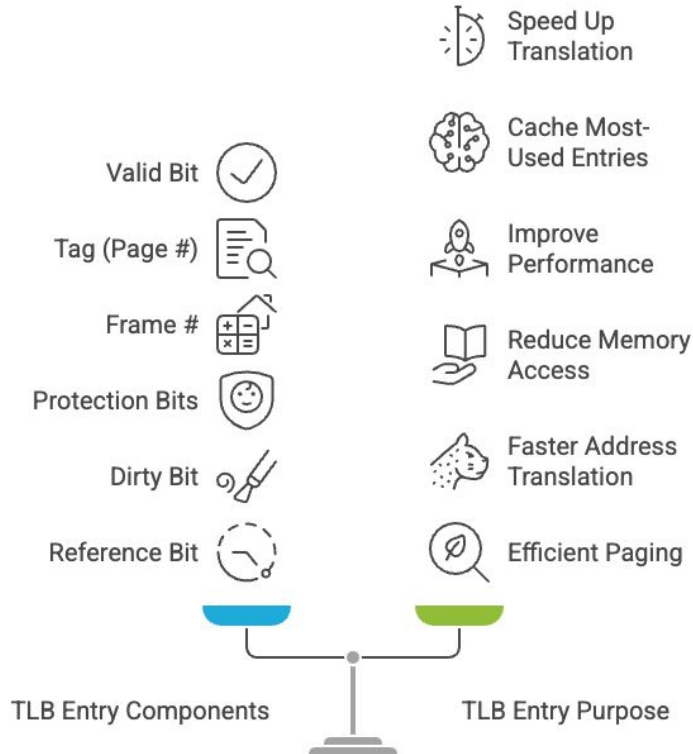
Translation Lookaside Buffer (TLB)

TLB Entry (What It Stores)

- **Valid bit** – entry usable or not
- **Tag (Page #)** – virtual page number
- **Frame #** – physical frame
- **Protection bits** – R / W / X
- **Dirty bit** – page modified?
- **Reference bit** – recently used?

Purpose

Speed up address translation by caching the most-used page table entries.



TLB Hit and Miss Scenarios

⚡ Scenario 1: TLB Hit (Fast Path)

- CPU generates virtual address (Page **P**, Offset)
- TLB searched **in parallel**
- ✓ Match found → Frame **F**
- Physical Address = (F, Offset)
- Memory accessed directly

🕒 Time: ~1 ns

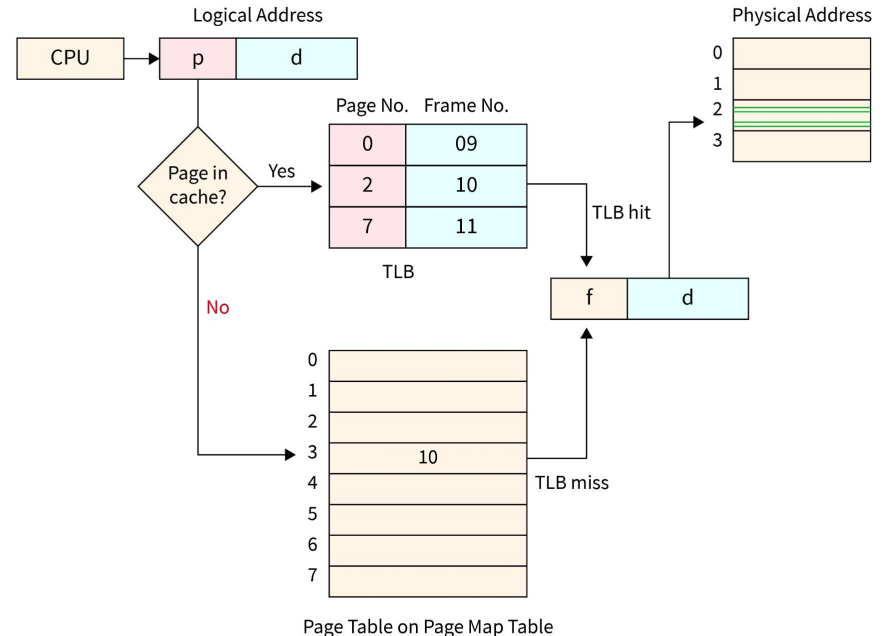
🐢 Scenario 2: TLB Miss (Slow Path)

- CPU generates virtual address (Page **P**, Offset)
- ✗ No TLB match
- Access **Page Table** in memory (via PTBR)
- Get Frame **F**
- Update TLB (may replace old entry)
- Access memory with physical address

🕒 Time: ~100+ ns (first access)

🔑 Key Takeaway

System performance depends heavily on **TLB hit rate**—hits are fast, misses are expensive.



TLB Performance Metrics



Key Terms

- **Hit Ratio (α):** Fraction of accesses found in TLB
- **Miss Ratio:** $1 - \alpha$
- **TLB Access Time (ϵ):** ~ 1 ns
- **Memory Access Time (m):** ~ 100 ns



Effective Access Time (EAT)

General Formula:

$$EAT = \epsilon + \alpha \cdot m + (1 - \alpha) \cdot (m + m)$$



What It Means

- **TLB Hit:** 1 memory access
- **TLB Miss:** 2 memory accesses (page table + data)
- Higher $\alpha \rightarrow$ EAT approaches m



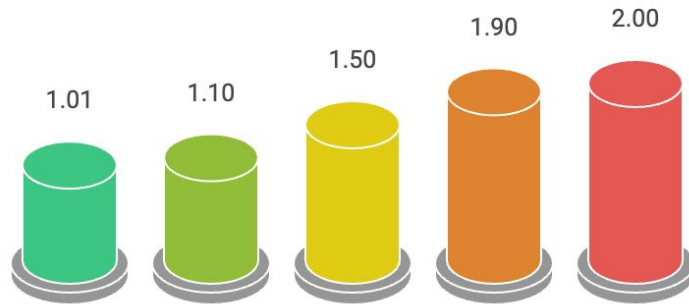
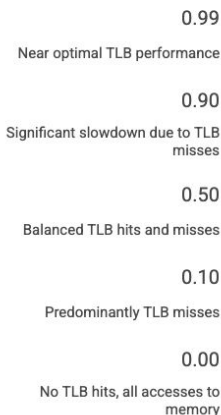
Example Insight

- $\alpha = 0.99 \rightarrow EAT \approx 1.01 \cdot m$ (near optimal)
- Small drops in α cause big slowdowns



Key Takeaway

High TLB hit ratio is crucial—it keeps paging performance close to raw memory speed.



TLB Performance Example

Given: TLB access (ϵ) ≈ 0 , Memory access (m) = 100 ns
and Hit ratio (α) = 0.98

EAT Calculation

$$\begin{aligned} \text{EAT} &= \alpha \cdot m + (1 - \alpha) \cdot 2m \\ &= 0.98 \cdot 100 + 0.02 \cdot 200 = 98 + 4 = 102 \text{ ns} \end{aligned}$$

Performance Impact

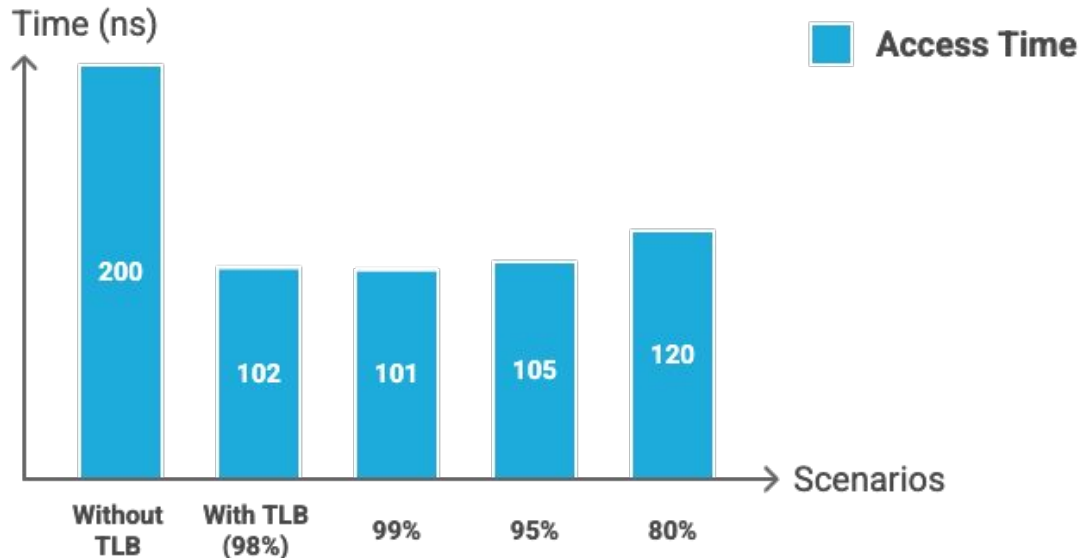
- Without TLB: 200 ns
- With TLB (98%): 102 ns
- Speedup: $\sim 1.96\times$

Hit Ratio Sensitivity

- 99% $\rightarrow$ 101 ns
- 95% $\rightarrow$ 105 ns
- 80% $\rightarrow$ 120 ns

Key Takeaway

Even a **small drop in TLB hit ratio** causes a **big performance hit**—high α is critical.



Practice Problems

Problem 2: TLB Performance:

A system uses a **Translation Lookaside Buffer (TLB)** to speed up address translation. The following parameters are given:

- **TLB hit ratio (α):** 96% (0.96)
- **TLB access time (ϵ):** 2 ns
- **Main memory access time (m):** 80 ns

Assume that:

- On a **TLB hit**, memory is accessed once.
- On a **TLB miss**, memory is accessed **twice** (once for page table lookup and once for data).

Calculate the **Effective Access Time (EAT)** of the system.

Calculation:

$$\text{EAT} = 2 + (0.96 \times 80) + (0.04 \times 160)$$

$$\text{EAT} = 2 + 76.8 + 6.4$$

$$\text{EAT} = 85.2 \text{ ns}$$

Explanation:

Most memory accesses are served directly from the TLB, so the average access time stays close to a single memory access.

Why TLB Works – Locality of Reference

Core Idea

TLB is effective because programs show **locality of reference**.

Spatial Locality

- Nearby memory locations accessed together
- Examples: sequential code, array traversal
- Many accesses fall **within the same page**
→ **One TLB entry covers many accesses**

Temporal Locality

- Recently used pages are reused soon
- Examples: loops, frequently called functions
- TLB keeps **recent translations**

Working Set

- Program uses a **small active set of pages**
- Working set usually **fits in the TLB**
- Typical hit ratios: **95–99%**

Example

- Loop: `array[0] ... array[999]`
- Page size = 4KB $\approx$ **1024 integers**
- **1 TLB miss $\rightarrow$ 999 hits**

Key Takeaway

High locality + small working set = **TLB hits most of the time**, making paging fast.

TLB Management Issues

Context Switching Problem

- Each process has its **own page table**
- On context switch, TLB may contain **old process entries**
-  Using them → **wrong physical memory access**

Solution 1: Flush TLB

- Invalidate all TLB entries on switch
- New process starts with **empty TLB**

✓ **Simple**

✗ **Cold start** → many TLB misses

Solution 2: ASID (Address Space ID)

- Tag each TLB entry with **Process ID**
- Format: | **ASID** | **Page #** | **Frame #** |
- Multiple processes coexist in TLB

✓ **No flush needed**

✓ **Faster context switches**

TLB Replacement Policies

🧠 When TLB Is Full

A new entry must replace an existing one — **which to evict?**

🔄 Common Policies

🕒 LRU (Least Recently Used)

- ✓ Best performance (uses locality)
- ✗ High hardware cost

🔄 NMRU (Not Most Recently Used)

- ✓ Near-LRU performance
- ✓ Lower complexity than LRU

📦 FIFO (First In First Out)

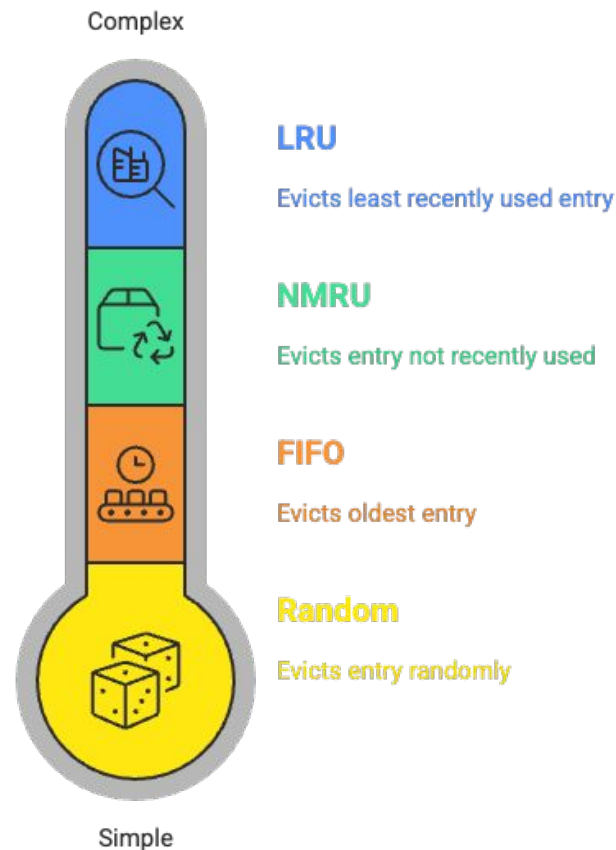
- ✗ May remove frequently used entries

🎲 Random

- Evict a random entry
- ✓ Very simple, low overhead

🔑 Key Takeaway

Hardware favors **simple, fast policies**—Random and NMRU are common, while true LRU is rare.



Introduction to I/O Management

🧠 What Is I/O Management?

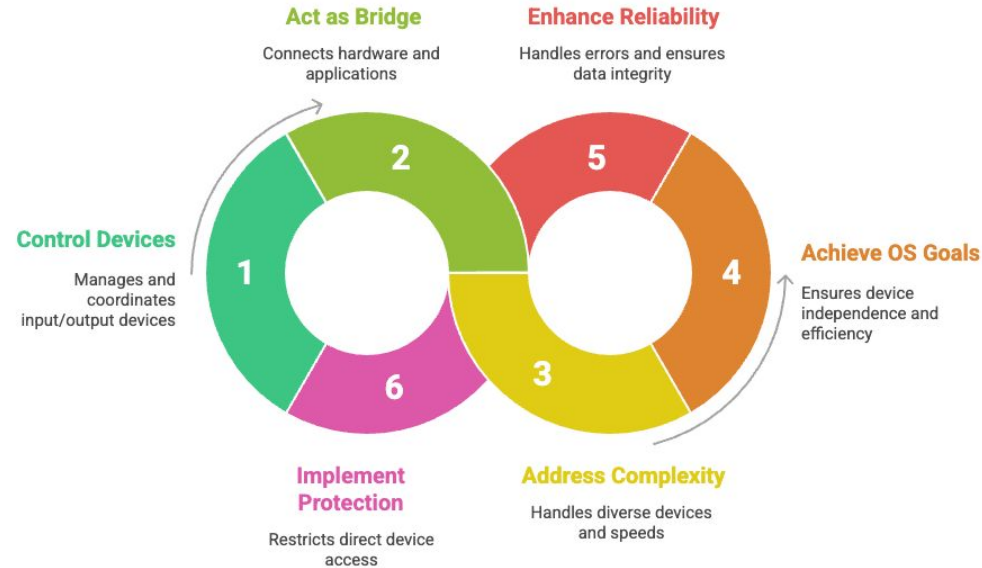
- Controls & coordinates **input/output devices**
- Acts as a **bridge** between hardware and applications
- One of the **most complex OS subsystems**

⚙️ Why It's Complex

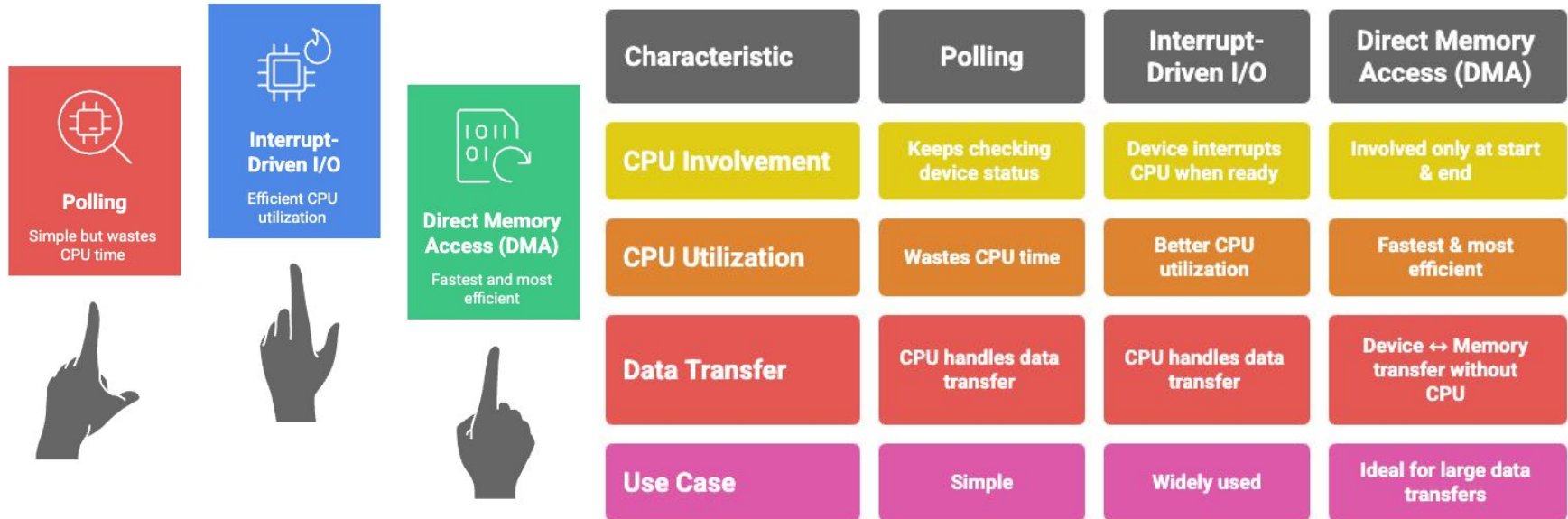
- Huge variety of devices
- Wide range of speeds (bytes/sec → GB/sec)
- Different protocols & data formats
- Real-time constraints
- Error handling & recovery

🎯 OS Goals for I/O

- **Device independence** – same API for all devices
- **Uniform naming** – consistent access
- **Efficiency** – low CPU overhead, high throughput
- **Reliability** – handle errors transparently
- **Protection** – restrict direct device access



I/O Communication Methods



Key Takeaway

Polling is simple, interrupts are efficient, DMA is optimal—modern systems rely heavily on interrupts and DMA.

Polling (Programmed I/O)

What It Is

CPU **continuously checks** device status until it's ready.

How It Works

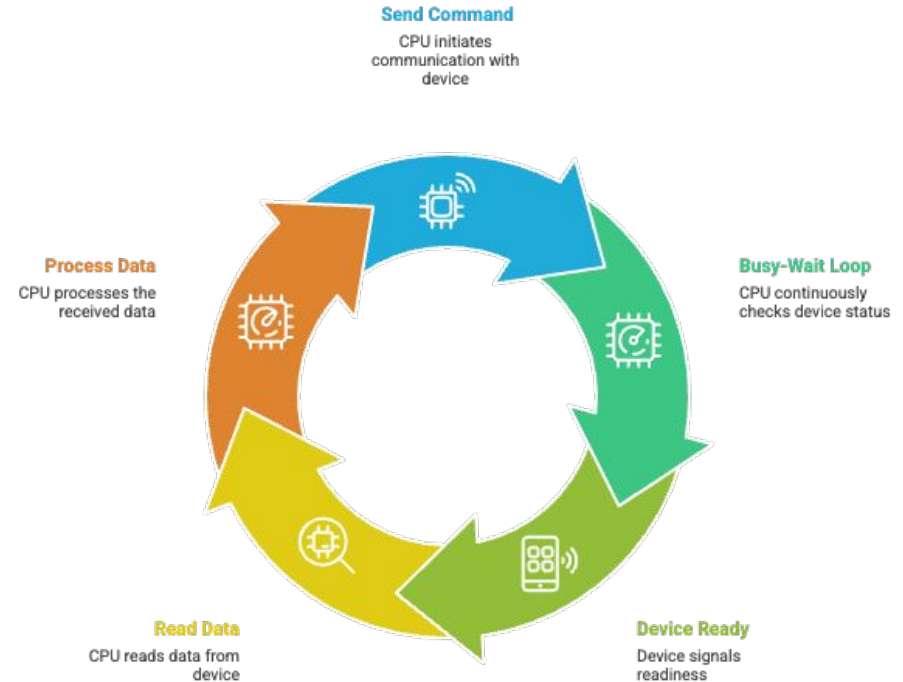
1. CPU sends command to device
2. **Busy-wait loop** checks status
3. Device becomes ready
4. CPU reads data and processes it

Simple Example

```
while (keyboard_status != READY) {  
    // busy wait  
}  
data = read_keyboard();
```

Characteristics

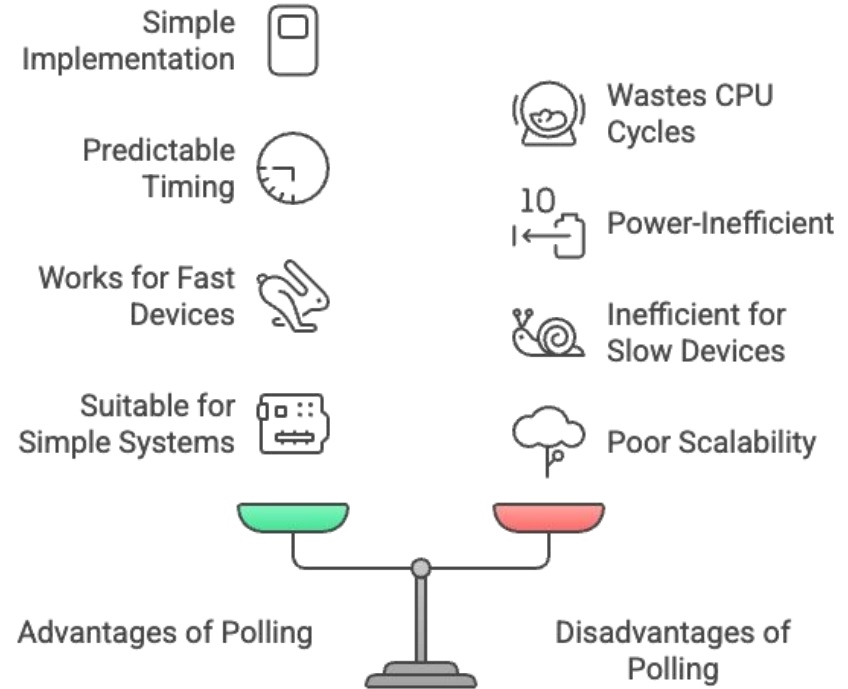
- ✓ Very simple
- ✓ No special hardware
- ✗ **Wastes CPU cycles**
- ✗ Poor efficiency



Polling – Advantages and Disadvantages

✚ When to Use Polling

- High-speed devices (negligible wait)
- Real-time systems with strict timing
- Small embedded systems (no interrupts)
- Acceptable if combined with **sleep/yield**



Interrupt-Driven I/O

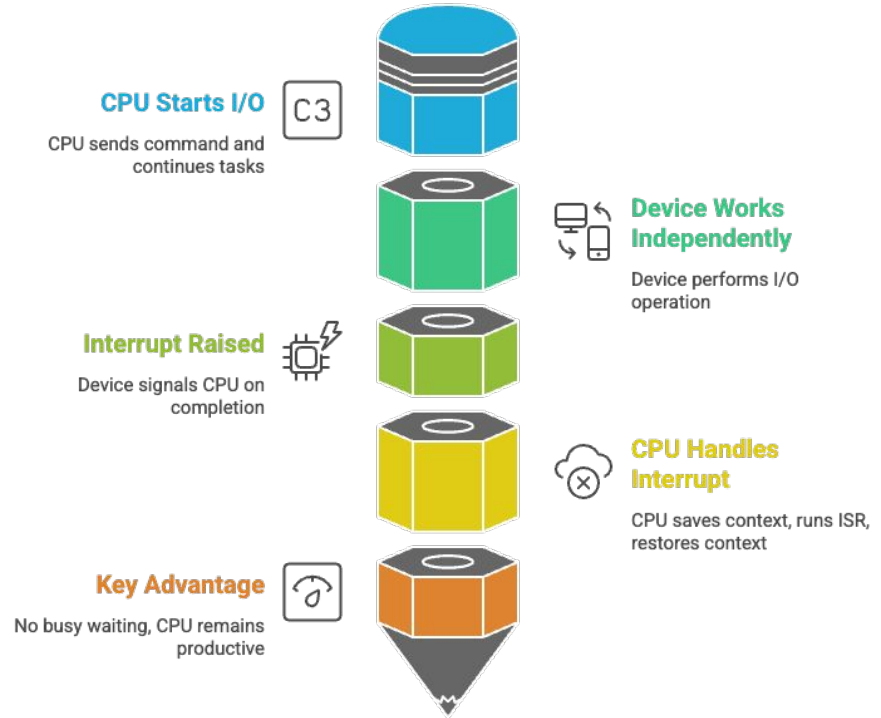
Device Signals CPU When Ready

How It Works

1. **CPU starts I/O**
 - Sends command to device
 - Continues executing other tasks
2. **Device works independently**
 - Performs I/O operation
3. **Interrupt raised**
 - Device signals CPU on completion
4. **CPU handles interrupt**
 - Saves context (PC, registers)
 - Runs **ISR** to read data
 - Restores context and resumes work

Key Advantage

- **No busy waiting**
- CPU remains productive while I/O runs.



Interrupt Processing Steps

How an Interrupt Is Handled (Step-by-Step)

1. **Device Ready**
 - Device controller raises an interrupt signal
2. **Interrupt Detection**
 - CPU detects interrupt at end of instruction cycle
3. **Interrupt Acknowledgment**
 - CPU sends ACK
 - Device places interrupt number on bus
4. **Context Save**
 - Save PC, registers, flags
 - Switch to **kernel mode**
5. **Interrupt Vector Lookup**
 - Use interrupt number to index **Interrupt Vector Table (IVT)**
 - Fetch ISR address
6. **Execute ISR**
 - Run device-specific handler
 - Read data / update buffers
7. **Context Restore**
 - Restore saved state
 - Resume interrupted process



Interrupt Vector Table

✖ What is IVT?

A table that maps **interrupt numbers** to their **Interrupt Service Routine (ISR) addresses**.

⚙ How It Works

1. Device raises interrupt with **interrupt number**
2. CPU uses number as **index into IVT**
3. Fetches ISR address
4. **Direct jump** to correct handler

★ Key Points

- One entry per interrupt
- Enables **fast ISR dispatch**
- Set up by OS at **boot time**
- Core to interrupt-driven I/O

📁 IVT Structure

Address	0x0000	0x0080	0x0084	0x0090	0x0094
IRQ #	0	32	33	36	37
Device	Divide Error	Timer	Keyboard	Disk	Network
ISR Address	0x1000	0x5000	0x5100	0x5400	0x5500

Interrupt Types and Priorities

By Source

Hardware Interrupts

- From I/O devices (keyboard, disk, timer)
- Asynchronous
- Signal external events

Software Interrupts

- Triggered by instructions (e.g., system calls)
- **Synchronous**
- OS service requests

Exceptions / Traps

- Program-generated errors
- Divide by zero, page fault, breakpoints

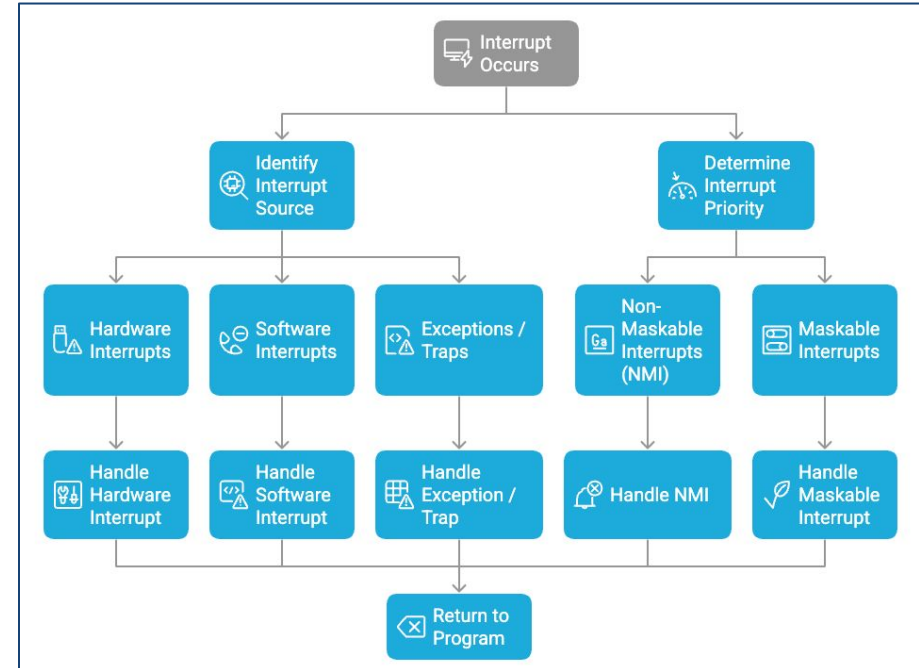
By Priority

Non-Maskable Interrupts (NMI)

- Cannot be disabled
- Critical events (hardware failure, power loss)

Maskable Interrupts

- Can be enabled/disabled
- Normal I/O devices

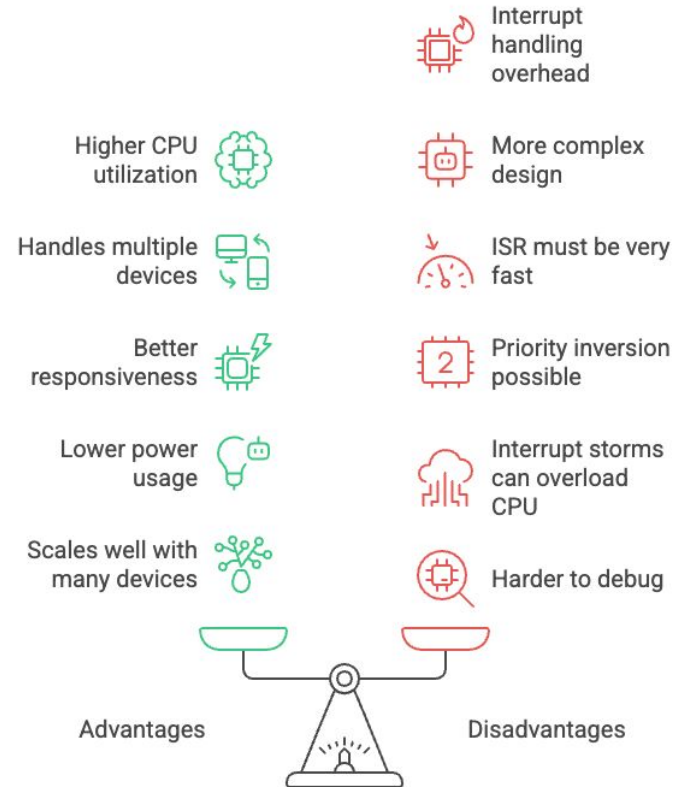


When multiple interrupts occur:  **Higher priority is handled first.**

Interrupt-Driven I/O – Advantages and Disadvantages

Typical Overhead

- Context save/restore: ~100 cycles
- ISR execution: minimal
- **Total:** ~1–10 μ s per interrupt



Problem with Interrupt-Driven I/O

The Data Transfer Bottleneck

Scenario: Read 1MB from Disk

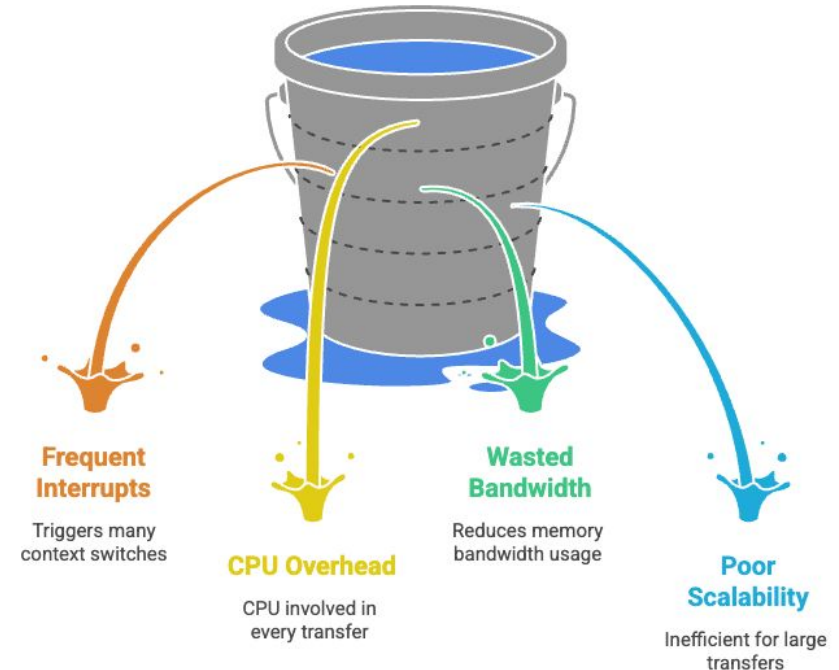
- Disk transfers **4 bytes** at a time
- Each transfer triggers an **interrupt**
- Total interrupts: **1MB / 4 bytes = 262,144 interrupts**

Why This Is a Problem

- CPU involved in **every tiny transfer**
- Massive **context switching**
- Wasted memory bandwidth
- CPU can't do useful work
- Poor scalability for large I/O

Key Insight

Interrupt-driven I/O works well for **small or infrequent data**, but becomes **inefficient for large transfers**.



Direct Memory Access (DMA)

🧠 What is DMA?

- Uses a **DMA Controller** (special hardware)
- Transfers data **directly between device and memory**
- **CPU involved only twice:**
 -  Start transfer
 -  Completion interrupt

⚙️ How DMA Works

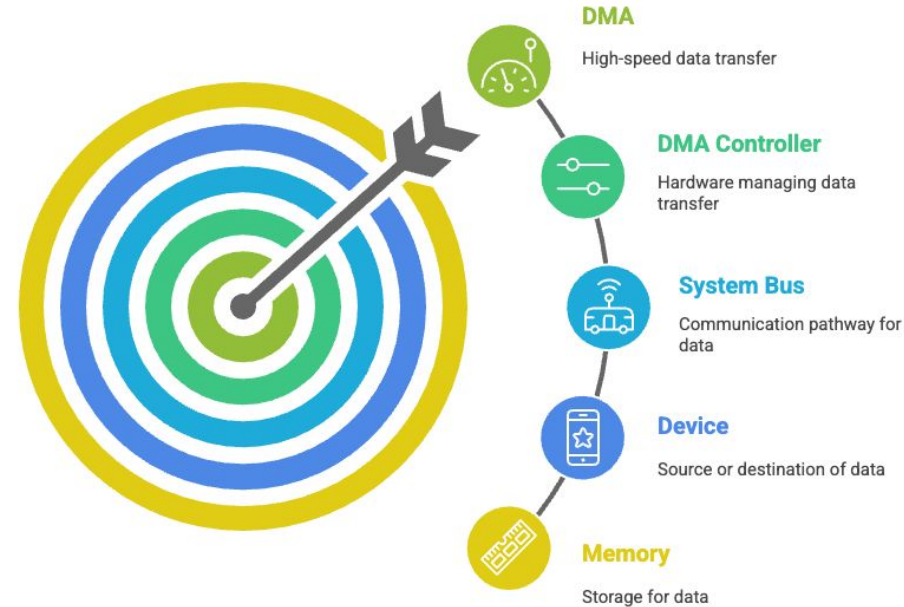
1. CPU sets up DMA (buffer address, size, direction)
2. DMA controller takes control of the system bus
3. Data moves **Device** $\rightleftharpoons$ **Memory** directly
4. DMA raises an interrupt when done

★ Key Idea

- DMA controller acts as a **bus master**
- CPU is **free to do other work** during transfer

🔑 One-Line Takeaway

DMA = High-speed data transfer with minimal CPU involvement



DMA Operation Steps

How a DMA Transfer Works

1 CPU Setup

- CPU programs DMA with:
 - Source address
 - Destination address
 - Transfer size
 - Mode (read/write)
-  CPU resumes other work

2 DMA Takes Control

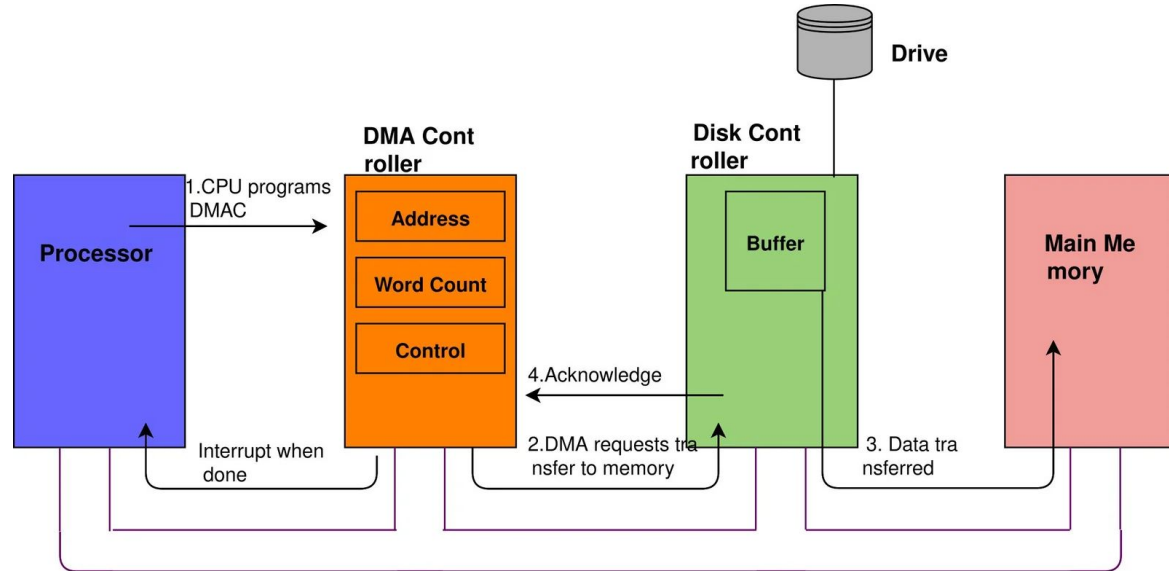
- DMA requests system bus
- CPU grants bus
-  DMA becomes **bus master**

3 Data Transfer

- DMA moves data **Device** ⇌ **Memory**
- Repeats until count = 0
-  No CPU involvement

4 Completion

- DMA releases bus
-  Interrupt sent to CPU
- CPU handles completion in ISR



DMA Transfer Modes

Characteristic	Bus Control	Transfer Speed	CPU Interference	Best Use Case
Burst Mode	Full control	Fastest	CPU paused	Large data
Cycle Stealing Mode	One cycle at a time	Balanced	CPU and DMA interleave	General-purpose
Transparent Mode	Only when CPU is idle	Slowest	No CPU interference	Background transfers

DMA vs CPU Transfer – Performance

 **Scenario: Transfer 1MB from disk → memory**

 1. CPU Programmed I/O (Polling)

- CPU moves **4 bytes at a time**
- **262,144 iterations**
-  **~500 ms**
-  CPU usage: **~100%**
-  **Extremely inefficient**

 2. Interrupt-Driven I/O

- **262,144 interrupts**
-  **~2 μs per interrupt**
-  **~500 ms interrupt overhead**
-  CPU partially blocked by interrupts

 3. Direct Memory Access (DMA)

- CPU setup: **~10 μs**
- DMA transfer: **~10 ms** (100 MB/s)
- Completion interrupt: **~2 μs**
-  **CPU involvement: ~12 μs total**
-  CPU **~99.9% free**

 Bottom Line

- **DMA reduces CPU overhead by ~40,000×**
- Best choice for **large data transfers**
- Polling & interrupts don't scale 

**CPU
Programmed
I/O (Polling)**

CPU polling is
extremely inefficient
with 100% CPU
usage.

**Direct Memory
Access (DMA)**

DMA transfers data
at 100 MB/s with
minimal CPU
involvement.

**Interrupt-
Driven I/O**

Interrupt-driven I/O
partially blocks the
CPU with frequent
interrupts.



DMA Advantages and Disadvantages

Characteristic	Advantage	Disadvantage
 CPU Involvement	Minimal	Bus contention
 Throughput	Very high	Overkill for small transfers
 Parallel Work	CPU can do other work	Cache coherency issues
 Interrupts	Dramatically fewer	Requires extra hardware
 I/O Suitability	Essential for high-bandwidth I/O	Not suitable for character devices
 Complexity	Less complex	More complex to program/debug

Best Use Cases

- Disk I/O, network packets
- Video/audio streaming
- Bulk transfers > **1KB**
- High-bandwidth peripherals

Key Takeaway:

DMA = maximum performance for large data, minimal CPU cost.

Practice Problems

Problem 4: DMA vs Interrupts:

A system transfers **10 MB of data** from disk to memory. The disk operates at a speed of **100 MB/s**. Compare the **CPU time consumed** in the following two I/O methods:

(A) Interrupt-driven I/O

- One interrupt is generated for every **512 bytes** transferred
- CPU time per interrupt = **3 μ s**

(B) Direct Memory Access (DMA)

- DMA setup time = **5 μ s**
- CPU time for completion interrupt = **3 μ s**

Calculate the **total CPU time** consumed in each case and determine which method is more efficient.

(A) Interrupt-Driven I/O

Interrupt frequency:

1 interrupt per 512 bytes

Total interrupts:

10 MB = $10 \times 1024 \times 1024 = 10,485,760$ bytes

Interrupts = $10,485,760 / 512 = 20,480$

CPU time per interrupt: 3 μ s

Total CPU time: $20,480 \times 3 \mu\text{s} = 61,440 \mu\text{s} = 61.44 \text{ ms}$

(B) DMA

DMA setup time: 5 μ s

Completion interrupt: 3 μ s

Total CPU time: $5 + 3 = 8 \mu\text{s}$

Explanation:

DMA transfers data without CPU involvement, making it vastly more efficient than interrupt-based transfers.

Cache Coherency Problem with DMA

Keeping CPU Cache & Main Memory Consistent

⚠ The Problem

DMA accesses **main memory directly**, bypassing CPU cache.

DMA Write (Device → Memory)

- DMA updates memory
- CPU cache still holds old value
- ❌ CPU reads **stale cached data**

DMA Read (Memory → Device)

- CPU updates cache only
- Memory not updated yet
- ❌ DMA reads **stale memory data**

🔧 Solutions

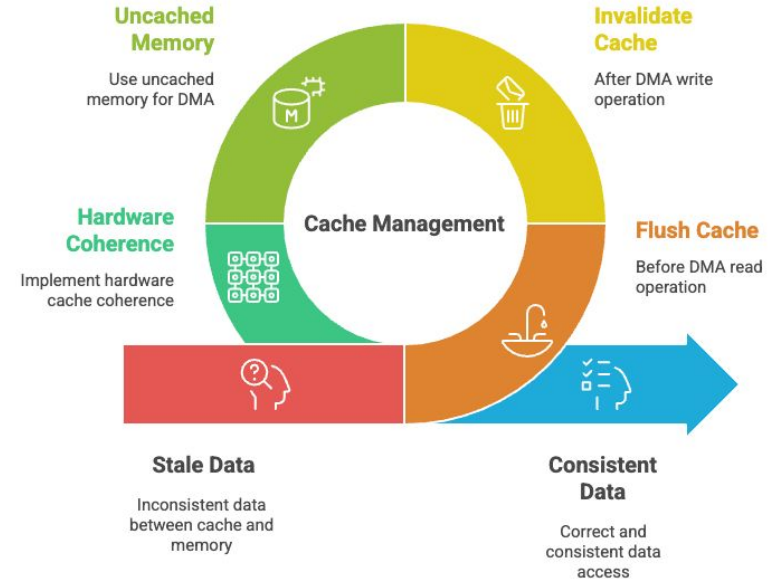
- 🔄 Flush cache before DMA read
- ❌ Invalidate cache after DMA write
- 🚫 Use uncached memory for DMA buffers
- 🧩 Hardware cache coherence

✅ Key Takeaway

DMA is fast but **unsafe without cache management**.

Proper cache control is essential for **correct and consistent data**.

Resolving Cache Coherency with DMA



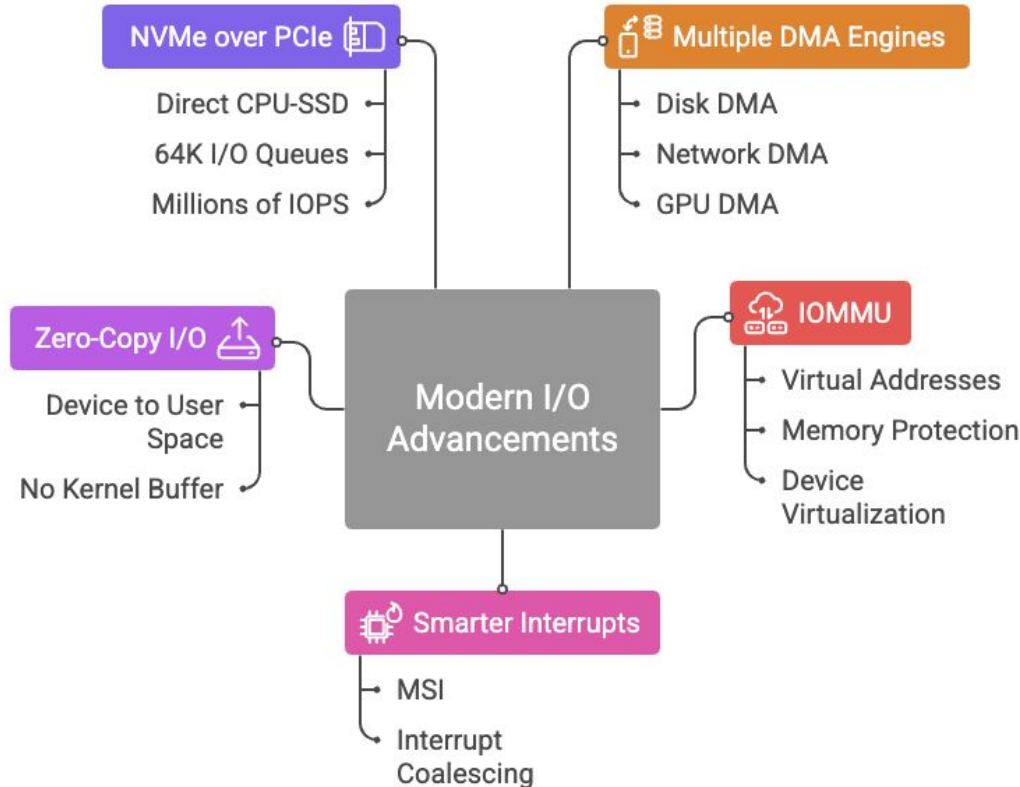
Comparison Summary – Polling vs Interrupts vs DMA

Aspect	Polling	Interrupts	DMA
 CPU Usage	Very High	Medium	Very Low
 Throughput	Low	Medium	High
 Complexity	Simple	Moderate	High
 Hardware Cost	Minimal	Low	Higher
 Response Time	Immediate	Fast (μ s)	Setup + interrupt
 Best Use Case	Very fast / simple devices	Character devices	Bulk data transfer
 CPU Overhead	~100% busy wait	Few μ s per interrupt	~10 μ s total
 Typical Devices	Sensors, microcontrollers	Keyboard, mouse	Disk, network, GPU

Key Takeaway

As data size and device speed increase, move from **Polling** → **Interrupts** → **DMA** for better performance and CPU efficiency.

Modern I/O Trends

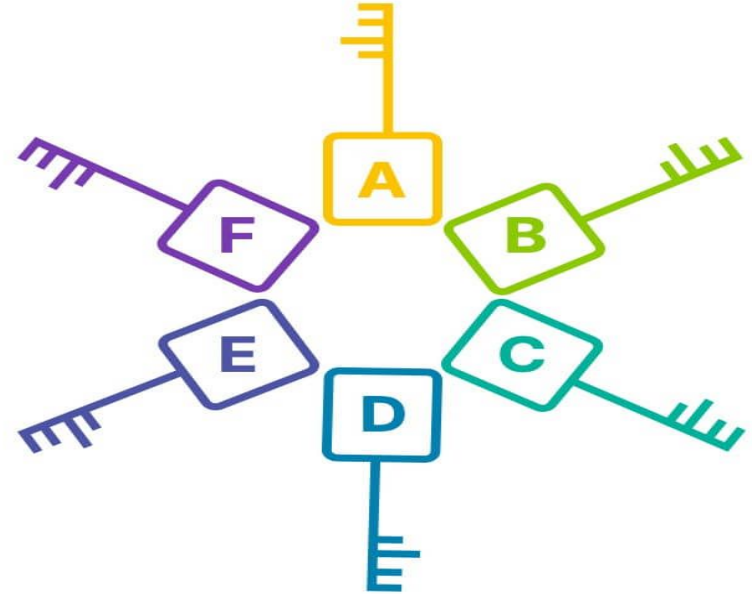


🔑 Key Takeaway

Modern I/O focuses on **parallelism**, **low latency**, **minimal CPU involvement**, and **high throughput** to match today's high-speed hardware.

Key Takeaways

- **Paging & Address Translation**
- **Translation Lookaside Buffer (TLB)**
- **Page Tables**
- **Polling I/O**
- **Interrupt-Driven I/O**
- **Direct Memory Access (DMA)**
- **Cache Coherency**
- **I/O Performance Comparison**
- **Modern I/O Optimizations**



Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.